Aksum University

AIT

Faculty Of Computing Technology

Department Of Computer Science

Group Assignment: Design And Analysis Of Algorithm

Q4 – Heap-Based Priority Queue: Emergency Room Triage System

No	NAME	ID-No
1	Weldearegay Gebrey	Aku-1602148
2	Eyerusalem Teklay	Aku-1602682
3	Shewit Legese	Aku-1602069

Emergency Room Triage System Using Heap-Based Priority Queue

Title Page

Course: Design and Analysis Of Algorithm
Project Title: Emergency Room Triage System Using Heap-Based Priority Queue
Algorithm: Max Heap Priority Queue

1. Introduction

Scheduling problems appear frequently in real-world systems where tasks must be processed according to priority. One critical example is **hospital emergency rooms (ER)**, where patients must be treated based on the severity of their medical condition rather than their arrival order.

For instance:

- A patient experiencing **cardiac arrest** must be treated immediately.
- A patient with **minor injuries** can wait longer.

If hospitals used a normal **First-Come First-Served (FCFS)** queue, less critical patients might receive treatment before critical ones, which could endanger lives.

To address this issue, hospitals use **triage systems** that assign priority levels to patients based on medical severity.

This project implements an **Emergency Room Triage System using a Max Heap Priority Queue**, where:

- Higher priority = more critical condition
- The most critical patient is always treated first

The system simulates a **24-hour emergency room operation** with **1,000 synthetic patients** and demonstrates the effectiveness of heap-based scheduling.

2. Problem Definition

The objective of this project is to design a system that efficiently manages patient treatment order in an emergency room.

Each patient record contains:

Attribute	Description
Patient ID	Unique identifier
Priority	Integer from 1–10 (10 = most critical)
Arrival Time	Timestamp when patient arrives

The system must support the following operations:

1. Insert

Adds a new patient to the queue.

```
insert(patientID, priority)
```

2. Extract Maximum

Removes and treats the most critical patient.

```
extractMax()
```

3. Increase Priority

Updates patient priority when condition worsens.

```
increasePriority(patientID, newPriority)
```

4. isEmpty

Checks whether the queue contains patients.

```
isEmpty()
```

3. Algorithm Design

The system is implemented using a **Max Heap**, a complete binary tree where every parent node has a value **greater than or equal to its children**.

The heap is stored using an **array representation**, which satisfies the project requirement of using arrays for data storage.

For an element at index **i**:

Parent index

$$Parent(i)=[(i-1)/2]$$

Left child index

$$Left(i)=2i+1$$

Right child index

$$Right(i)=2i+2$$

This array representation allows efficient navigation of the heap without using pointers.

4. Algorithm Operations

4.1 Insert Operation

When a patient arrives, they are inserted at the end of the heap and moved upward until the heap property is restored.

Steps

1. Insert patient at the end of the heap.
2. Compare with parent node.
3. Swap if priority is higher.
4. Continue until heap property is satisfied.

Pseudocode

```
INSERT(patient)
    size ← size + 1
    i ← size

    while i > 0 AND heap[i] > heap[parent(i)]
        swap(heap[i], heap[parent(i)])
        i ← parent(i)
```

4.2 ExtractMax Operation

The most critical patient is always stored at the **root of the heap**.

Steps

1. Remove the root node.
2. Replace it with the last element.
3. Apply maxHeapify to restore heap property.

Pseudocode

```
EXTRACT_MAX()
    if heap is empty
        return null
     max ← heap[0]
    heap[0] ← heap[size]
    size ← size – 1
    MAX_HEAPIFY(0)

    return max
```

4.3 Increase Priority Operation

If a patient's condition worsens, their priority must be updated.

Steps

1. Locate patient in heap
2. Update priority
3. Move upward until heap property is restored

Pseudocode

```
INCREASE_PRIORITY(id, newPriority)

    locate patient index i

    if newPriority <= heap[i].priority
        return

    heap[i].priority ← newPriority

    while i > 0 AND heap[i] > heap[parent(i)]
        swap(heap[i], heap[parent(i)])
        i ← parent(i)
```

4.4 MaxHeapify Function

Maintains heap property after extraction.

```
MAX_HEAPIFY(i)

    largest ← i
    left ← 2i + 1
    right ← 2i + 2

    if left < size AND heap[left] > heap[largest]
        largest ← left

    if right < size AND heap[right] > heap[largest]
        largest ← right

    if largest ≠ i
```

```
        swap(heap[i], heap[largest])
        MAX_HEAPIFY(largest)
```

5. Dataset

The system uses a **synthetic dataset of 1000 patients** stored in a CSV file.

Example dataset format:

PatientID	Priority	ArrivalTime
1	7	10:03:22
2	4	10:05:41
3	10	10:06:15
4	2	10:07:09

Dataset Generation

The dataset was generated using random functions:

- Patient ID: 1 – 1000
- Priority: Random number from 1 – 10
- Arrival Time: Random timestamp

CSV file example:

PatientID	,Priority	ArrivalTime
1	7	10:03:22
2	4	10:05:41
3	10	10:06:15

6. Simulation

A **24-hour emergency room simulation** was conducted.

Simulation Procedure

1. Load 1000 patients from CSV

2. Build the max heap using `buildMaxHeap()`
3. Simulate patient treatment every few minutes
4. Insert new arriving patients
5. Occasionally increase patient priority

Example Output

```
New PatientID 152 arrived | Priority 6 | Arrival 09:12:14

Treating PatientID 87 | Priority 10 | Arrival 08:55:11
Treating PatientID 213 | Priority 9 | Arrival 09:01:30
Treating PatientID 45 | Priority 8 | Arrival 09:03:18

Condition worsened → PatientID 76 new priority 9
```

This output confirms that **patients with higher priority are treated first**, regardless of arrival order.

7. Complexity Analysis

Insert Operation

During insertion, the element moves upward along the heap height.

Height of heap:

$h=(\log_{10} n)$

Therefore:

Time Complexity:

$O(\log_{10} n)$

ExtractMax Operation

After removing the root, the heap property must be restored using `maxHeapify`.

Worst-case complexity: O(logn)

Build Heap Operation

Using **Floyd's Heap Construction Algorithm**, the heap can be built from an unsorted array.

Although heapify is O(logn), it is applied mostly to nodes near the bottom.

Total complexity: O(n)

Space Complexity

The heap is stored in an array.

If **n = number of patients**:

Space complexity: O(n)

8. Experimentation and Results

The system was tested using **1000 patient records**.

Observations

Test Scenario	Result
High priority patient arrives late	Treated immediately
Same priority patients	Ordered by arrival time
Priority update	Patient moves up in heap

Key Findings

- Patients with **priority 10 were always treated first**
- System efficiently handled **large datasets**
- Heap operations maintained correct ordering

9. Real-World Applications

Heap-based priority queues are widely used in many systems:

- Emergency room triage systems
- CPU scheduling
- Network packet scheduling
- Air traffic control
- Event-driven simulations

These applications require **efficient priority-based task management**.

10. Conclusion

This project designed and implemented an **Emergency Room Triage System using a Max Heap Priority Queue**.

The system ensures that patients with the most critical conditions receive treatment first, regardless of arrival order.

The simulation with **1000 synthetic patients** demonstrated the efficiency and practicality of heap-based priority scheduling.

Key results:

- Insert and extract operations run in **O(log n)** time
- Heap construction runs in **O(n)** time
- The system efficiently manages large-scale scheduling problems

This project highlights how **algorithm design directly impacts real-world decision-making systems**.

11. References

1. Cormen, T. H. *Introduction to Algorithms*, MIT Press

2. Kleinberg, J., Tardos, É. *Algorithm Design*, Pearson
3. Kaggle Dataset Repository – https://www.kaggle.com
4. GeeksforGeeks – Heap Data Structure Tutorials